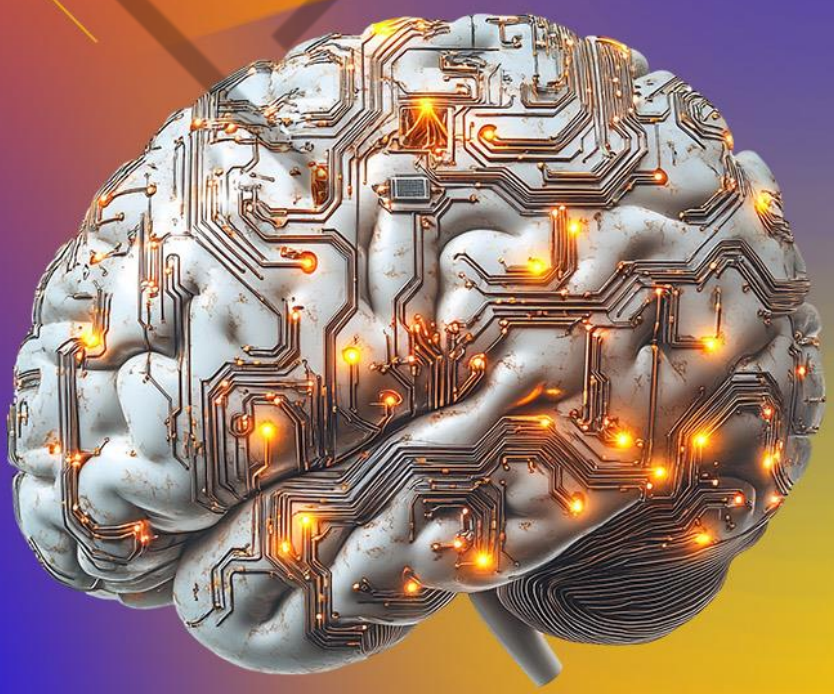
FIAP ON

COLHEITA DE SOLUÇÕES INTELIGENTES - TRANSFORMANDO DADOS EM AÇÕES

O OLHAR DIGITAL: TÉCNICAS DE DETECÇÃO E SEGMENTAÇÃO



10

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Diferença entre classificação, detecção e segmentação 5
Figura 2 - Algumas aplicações de detecção e segmentação de objetos 7
Figura 3 - Labels de detecção (*bounding box*) e segmentação (máscara), respectivamente 9
Figura 4 - Cálculo da métrica IoU 11
Figura 5 - Interpretação da métrica IoU 11
Figura 6 - Interpretação gráfica de precision e recall 12
Figura 7 - Interpretação gráfica do Dice Coefficient 14
Figura 8 - Padrões típicos buscados pelo algoritmo de Haar Cascades 18
Figura 9 - Exemplo de descritor de features HOG na imagem 20
Figura 10 - Principais arquiteturas de Deep Learning para processamento de imagens 23
Figura 11 - Arquitetura R-CNN 24
Figura 12 - Comparação das arquiteturas R-CNN, Fast R-CNN e Faster R-CNN 25
Figura 13 - Intuição da arquitetura YOLO 26
Figura 14 - Comparação da YOLOv3 com a YOLOv4 na base MS COCO 29
Figura 15 - Tipos de segmentação 30
Figura 16 - Arquitetura U-Net 32
Figura 17 - Arquitetura SAM 33
Figura 18 - Exemplo de segmentação com SAM 35
Figura 19 - Saída da execução do código-fonte 3 39
Figura 20 - Imagem de entrada para SAM 40
Figura 21 - Saída do código-fonte 7 43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Instalações necessárias para YOLO 37
Código-fonte 2 - Imports e método de exibição de resultados 38
Código-fonte 3 - Usando a YOLO para detectar objetos e exibindo os resultados ...38
Código-fonte 4 - Instalações necessárias para SAM.....39
Código-fonte 5 - Imports e carregamento da imagem de entrada.....40
Código-fonte 6 - Usando a biblioteca SAM.....41
Código-fonte 7 - Exibindo o resultado da segmentação pela biblioteca SAM 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 O OLHAR DIGITAL: TÉCNICAS DE DETECÇÃO E SEGMENTAÇÃO 5
1.1 Diferença entre classificação, detecção e segmentação.....5
1.2 Aplicações reais 6
2 AVALIAÇÃO DA QUALIDADE DOS MÉTODOS.....9
2.1 Métricas de detecção 10
2.2 Métricas de segmentação 13
3 CRIAÇÃO DE LABELS PARA DETECÇÃO E SEGMENTAÇÃO.....15
4 MÉTODOS CLÁSSICOS.....17
4.1 Algoritmo de Haar Cascades.....17
4.1.1 Como o Haar Cascades funciona?.....18
4.1.2 Aplicações e limitações 19
4.2 Algoritmo HOG + SVM 20
4.2.1 Como o HOG + SVM funciona? 20
4.2.2 Aplicações e limitações 21
4.3 Limitações dos métodos tradicionais.....22
5 REDES NEURAIS PARA DETECÇÃO DE OBJETOS.....24
5.1 Breve histórico.....24
5.2 Arquitetura YOLO.....25
5.2.1 Detalhamento 26
5.2.2 Evolução em suas versões.....28
6 REDES NEURAIS PARA SEGMENTAÇÃO DE OBJETOS 30
6.1 Tipos de segmentação 30
6.2 Breve histórico.....31
6.3 Arquitetura SAM 32
6.3.1 Detalhamento 34
6.3.2 O impacto do SAM na visão computacional 35
7 HANDS ON: DETECTANDO E SEGMENTANDO OBJETOS.....37
7.1 Detecção de objetos com YOLO 37
7.2 Segmentação de objetos com SAM 39
CONCLUSÃO.....44
REFERÊNCIAS.....45
GLOSSÁRIO 47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 O OLHAR DIGITAL: TÉCNICAS DE DETECÇÃO E SEGMENTAÇÃO

1.1 Diferença entre classificação, detecção e segmentação

Na visão computacional, classificação, detecção e segmentação são tarefas distintas, mas frequentemente complementares no processamento e na análise de imagens. Cada uma dessas abordagens tem um propósito específico e um nível crescente de complexidade na extração de informações visuais. Vamos conhecer estas diferenças, ilustradas na figura a seguir.

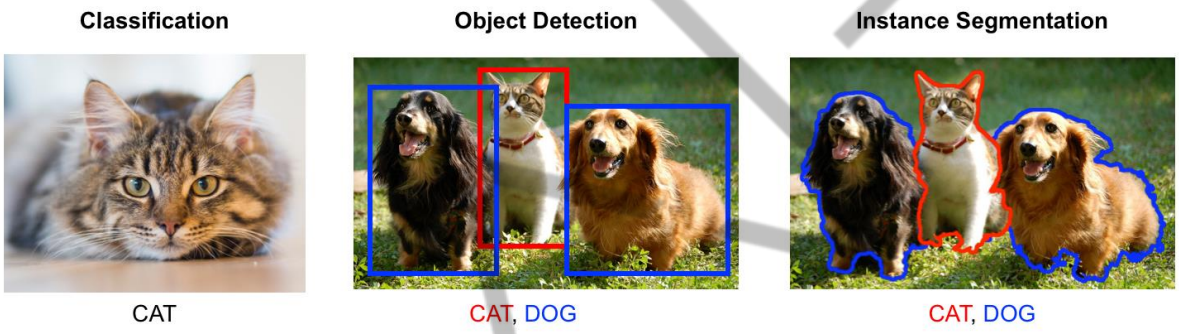


Figura 1 - Diferença entre classificação, detecção e segmentação
Fonte: Medium (2018)

- **Classificação de objetos:** a classificação é a tarefa mais básica e consiste em **identificar qual objeto está presente em uma imagem**. O modelo recebe uma imagem inteira e atribui a ela um rótulo de uma classe pré-definida. Por exemplo, um classificador de imagens treinado para distinguir entre cães e gatos indicaria se uma imagem contém um cão ou um gato, mas sem indicar sua posição exata. Modelos como ResNet, VGG e EfficientNet são amplamente utilizados para essa finalidade.
- **Detecção de objetos:** a detecção vai além da classificação, pois além de identificar quais objetos estão na imagem, também **determina onde eles estão**. Isso é feito através da predição de caixas delimitadoras (*bounding boxes*) ao redor dos objetos. A detecção é essencial para aplicações como monitoramento de segurança, carros autônomos e reconhecimento facial. Arquiteturas como Faster R-CNN, YOLO (You Only Look Once) e SSD

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

(Single Shot Detector) são exemplos de modelos avançados usados para essa tarefa.

- **Segmentação de objetos:** a segmentação leva a detecção um passo além, pois não apenas localiza o objeto, mas também **define sua forma exata em nível de pixel**. Veremos adiante que existem dois tipos principais de segmentação: a **segmentação semântica** (exemplo: em uma foto de estacionamento detectar e diferenciar carros e pedestres) e a **segmentação de instâncias** (detectar cada carro individualmente).

Portanto, enquanto a classificação apenas identifica o objeto, a detecção localiza sua posição na imagem, e a segmentação separa cada objeto com precisão de pixel. Quanto mais detalhada a análise, maior a complexidade computacional envolvida, tornando a escolha do modelo ideal essencial para cada aplicação.

1.2 Aplicações reais

A detecção e segmentação de objetos têm se tornado fundamentais em diversas indústrias, permitindo que máquinas entendam e interpretem imagens com alta precisão. Essas tecnologias são utilizadas em segurança, saúde, indústria, mobilidade e entretenimento, otimizando processos e aprimorando a tomada de decisão em tempo real. A figura a seguir ilustra algumas delas:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

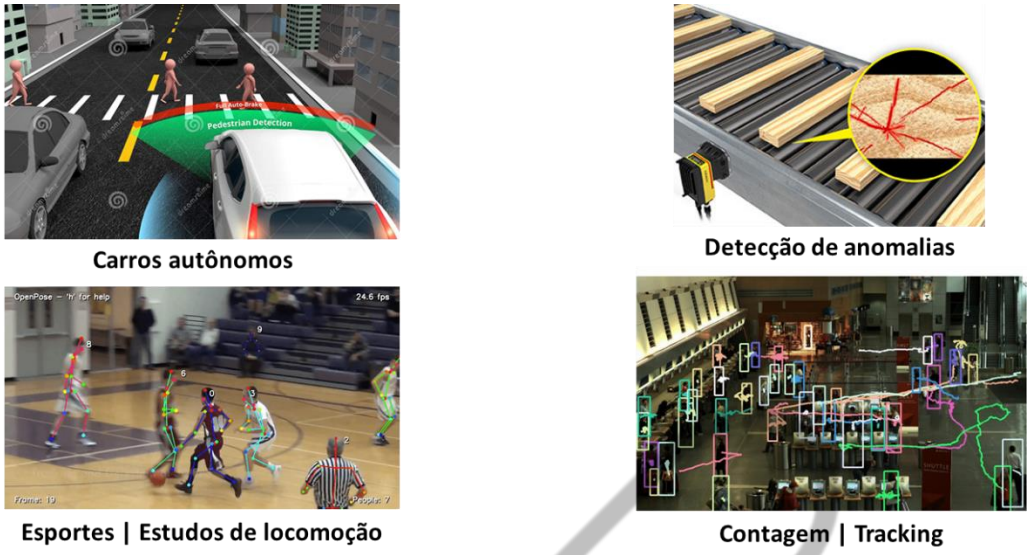


Figura 2 - Algumas aplicações de detecção e segmentação de objetos
Fonte: Elaborado pelo autor (2025)

- **Segurança e vigilância:** sistemas de monitoramento inteligente utilizam detecção de objetos para **reconhecer rostos**, **identificar comportamentos suspeitos** e **detectar intrusos** em áreas restritas. Câmeras com visão computacional podem diferenciar humanos de objetos inanimados, alertando automaticamente sobre atividades incomuns. A segmentação de objetos aprimora esse processo ao permitir que sistemas identifiquem ações específicas, como alguém deixando um pacote em um local público por um período prolongado.

- **Medicina e diagnóstico por imagem:** na área da saúde, a segmentação de objetos é essencial em diagnósticos médicos assistidos por IA, como na detecção de tumores, lesões e órgãos em exames de imagem. Técnicas como U-Net e Mask R-CNN são amplamente utilizadas para **segmentação de tomografias**, **ressonâncias magnéticas** e **radiografias**, permitindo análises mais detalhadas e auxiliando médicos na identificação precoce de doenças como câncer, Alzheimer e COVID-19.

- **Veículos autônomos e mobilidade:** a detecção de objetos é um dos pilares da **navegação autônoma**, permitindo que **veículos identifiquem pedestres, semáforos, placas de trânsito e outros carros em tempo real**. Sensores LiDAR e câmeras utilizam técnicas como YOLO e Faster R-CNN para fornecer ao carro uma visão detalhada do ambiente ao seu redor. Em relação a segmentação, ela é usada para separar faixas de pedestres,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

áreas de calçadas e limites de pistas, garantindo maior segurança na condução autônoma.

- **Indústria e automação:** na indústria, a detecção e a segmentação de objetos são usadas em inspeção de qualidade e automação de produção. As máquinas inteligentes podem **detectar defeitos em peças**, **classificar produtos** e **monitorar processos fabris** com extrema precisão. No setor agrícola, drones equipados com IA podem **detectar pragas** e **deficiências nutricionais em plantações**, otimizando o uso de fertilizantes e pesticidas, promovendo uma agricultura mais sustentável e eficiente.
- **Entretenimento e realidade aumentada:** em aplicações interativas, como filtros de redes sociais, games e realidade aumentada, a **segmentação de objetos permite efeitos visuais que se ajustam dinamicamente ao ambiente**. Ferramentas como Google Lens e aplicativos de provadores virtuais utilizam essas técnicas para identificar produtos e fornecer informações em tempo real. Na produção audiovisual, **segmentação de imagem permite efeitos especiais**, como substituição de fundos sem necessidade de *chroma key*.

O impacto da detecção e segmentação de objetos continuará crescendo à medida que a IA evolui, abrindo novas possibilidades para inovação e otimização de processos em diferentes setores.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 AVALIAÇÃO DA QUALIDADE DOS MÉTODOS

Assim como todo processo informatizado, a avaliação da qualidade da resposta é fundamental para termos garantias do funcionamento do modelo. Avaliar corretamente os modelos de detecção e segmentação de objetos é essencial para garantir sua precisão, confiabilidade e aplicabilidade no mundo real.

A avaliação pressupõe um aprendizado supervisionado e a existência de labels para as tarefas. No caso, os labels de detecção e segmentação são mais complexos do que apenas uma string, como na classificação. Precisamos de ***bounding boxes*** (para detecção) ou **máscaras** (para segmentação). Na figura "Labels de detecção (*bounding box*) e segmentação (máscara), respectivamente", o label para classificação da imagem pode ser "cachorro"; o label da detecção é um retângulo que posiciona onde o cão está na imagem; o label para a segmentação é uma máscara que indica quais pixels pertencem ao cão e quais são background da imagem.

Figura 3 - Labels de detecção (*bounding box*) e segmentação (máscara), respectivamente
Fonte: Roboflow blog (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Sem métricas bem definidas, um modelo pode parecer eficiente em um ambiente controlado, mas falhar gravemente em situações reais, comprometendo segurança, desempenho e tomadas de decisão. Em sistemas críticos, como veículos autônomos e diagnóstico médico, um erro na detecção pode ter consequências graves, colocando vidas em risco.

Além disso, métricas inadequadas podem ocasionar otimizações equivocadas, resultando em redes neurais que parecem funcionar bem, mas apresentam baixos índices de generalização para novos dados. Definir corretamente indicadores de desempenho permite comparar modelos, ajustar hiperparâmetros e garantir que a solução seja robusta o suficiente para seu contexto de aplicação.

Para classificação de imagens, métricas como acurácia, área abaixo da curva e matriz de confusão fazem sentido. Mas quais são as métricas pertinentes nos cenários de detecção e classificação? Antes de respondermos esta questão, é importante distinguir os **objetivos da avaliação**:

- **Detecção de objetos:** mensura o posicionamento e classificação (*bounding boxes*).
- **Segmentação de objetos:** mensura precisão pixel a pixel (máscaras).
- Algumas métricas podem ser aplicadas a ambas as tarefas, mas com interpretações distintas.

Assim como na classificação, para avaliar um modelo de detecção e segmentação, é ideal combinar múltiplas métricas para capturar nuances diferentes da sua interpretação e da corretude do modelo.

2.1 Métricas de detecção

A avaliação de modelos de detecção de objetos exige métricas específicas para medir a qualidade das previsões em termos de **localização**, **classificação** e **abrangência dos objetos detectados**. Dado que o objetivo da detecção é prever corretamente *bounding boxes* ao redor dos objetos e atribuir rótulos adequados a cada um deles, diferentes métricas foram desenvolvidas para quantificar o desempenho dos

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

modelos. Entre as principais, destacam-se **Intersection over Union (IoU), Mean Average Precision (mAP)**, **Recall**, **Precision** e **F1-Score**.

A **Intersection over Union (IoU) mede a sobreposição entre a caixa predita e a caixa real (*ground truth*)**. O cálculo é realizado dividindo a interseção das áreas das duas caixas pelo total da união entre elas. Observe a figura:

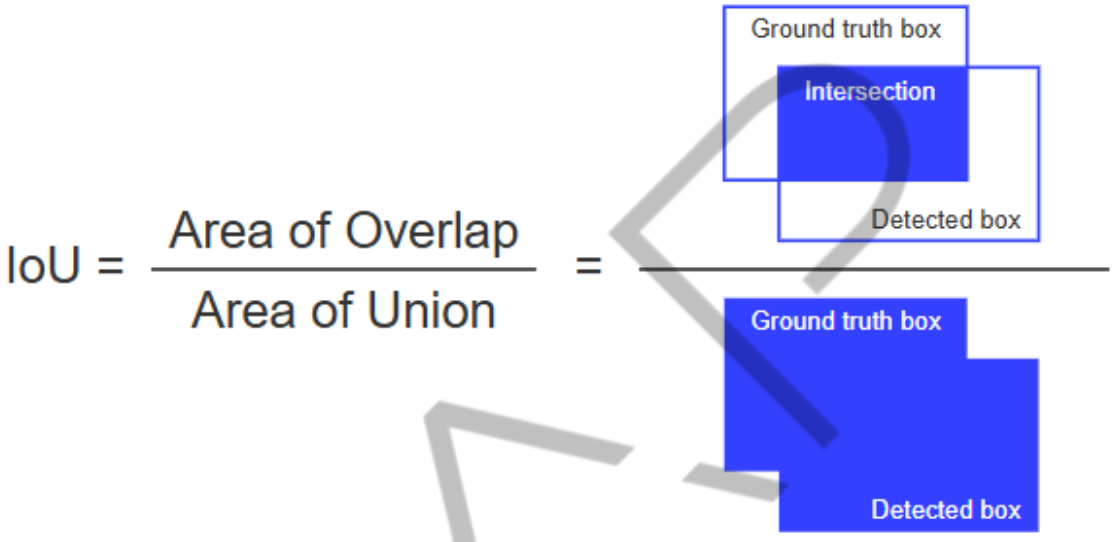


Figura 4 - Cálculo da métrica IoU
Fonte: Baeldung (2024), adaptado por FIAP (2025)

O valor resultante varia entre 0 e 1, onde 1 indica uma sobreposição perfeita e valores mais baixos indicam desvios entre a previsão e a posição real do objeto (figura “Interpretação da métrica IoU”). Modelos de detecção geralmente utilizam um limiar de IoU ≥ 0.5 para considerar uma predição como correta, embora aplicações mais exigentes, como detecção médica ou industrial, possam requerer IoU ≥ 0.75.

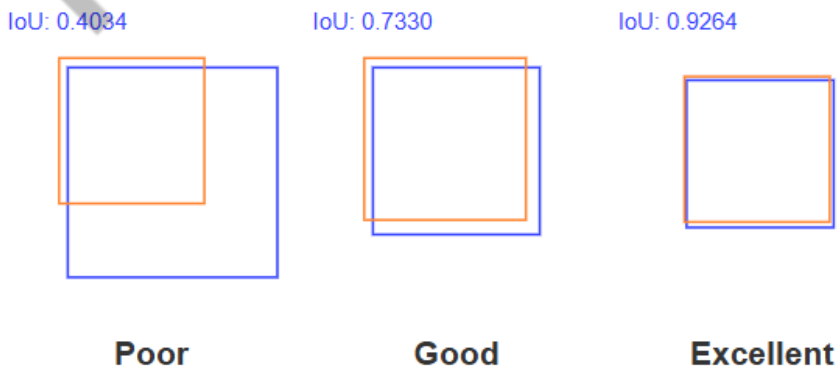


Figura 5 - Interpretação da métrica IoU
Fonte: PyImageSearch (2016), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A **Average Precision (AP) mede a qualidade da detecção considerando a precisão ao longo de diferentes limiares de confiança do modelo**. A curva Precision-Recall é utilizada para calcular a área sob a curva (AUC), e o AP é obtido como a média dessas precisões para diferentes pontos de recall. O **Mean Average Precision (mAP)** é a **média da AP calculada para todas as classes de objetos no conjunto de dados**. Valores de mAP acima de 0.5 são considerados bons em aplicações comuns, enquanto mAP superiores a 0.75 indicam um modelo altamente preciso.

A **precisão (Precision) mede a proporção de objetos detectados que são realmente verdadeiros positivos**. Um modelo com alta precisão tem menos predições erradas (falsos positivos). Já o **recall mede quantos dos objetos reais foram efetivamente detectados**. Um modelo com alto recall identifica a maioria dos objetos, mas pode ter mais falsos positivos. O equilíbrio entre essas duas métricas é representado pelo F1-Score, que é a média harmônica entre precisão e recall (figura “Interpretação gráfica de precision e recall”). O ideal é que um modelo atinja F1-Score acima de 0.7 para garantir um bom desempenho sem comprometer a detecção excessiva ou insuficiente.

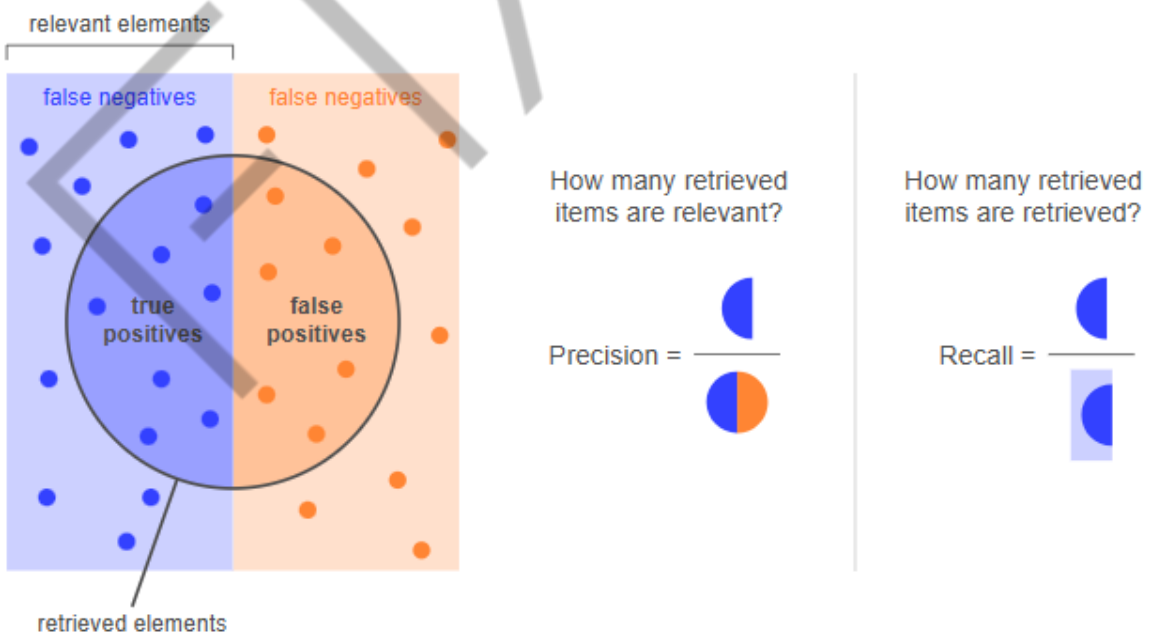


Figura 6 - Interpretação gráfica de precision e recall
Fonte: SuperAnnotate (2023), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No contexto de aplicações práticas, métricas como mAP@50 e mAP@75 são frequentemente reportadas. O **mAP@50** refere-se à média de precisão considerando um limiar de IoU ≥ 0.5, enquanto o **mAP@75** considera um limite mais rigoroso (IoU ≥ 0.75). Modelos avançados, como os utilizados em visão computacional para veículos autônomos, tendem a apresentar mAP@75 acima de 0.7, garantindo uma precisão mais alta na localização dos objetos.

2.2 Métricas de segmentação

A segmentação de imagens é um dos desafios mais complexos da visão computacional, pois envolve a classificação de cada pixel da imagem em uma determinada categoria, exigindo métricas específicas para avaliar o desempenho do modelo. Diferente da detecção de objetos que trabalha com *bounding boxes*, a segmentação precisa garantir que a previsão siga com precisão os contornos e a área real dos objetos. Para isso, algumas das métricas mais utilizadas incluem **Intersection over Union (IoU)**, **Dice Coefficient**, **Pixel Accuracy** e **Boundary F1 Score**.

A **Intersection over Union (IoU)**, também conhecida como **Jaccard Index**, possui o mesmo cálculo e interpretação do cenário de detecção, porém mede a sobreposição entre a máscara predita e a máscara real.

O **Dice Coefficient**, também chamado de Sorensen-Dice Score, é semelhante ao IoU, mas coloca mais peso nos pixels de interseção. Seu cálculo considera o dobro da área de interseção dividido pela soma das áreas predita e real (figura “Interpretação gráfica do Dice Coefficient”). Ele também varia entre 0 e 1, e quanto mais próximo de 1, melhor. Para segmentações eficientes, espera-se um Dice Coefficient superior a 0.7, e modelos de alto desempenho frequentemente atingem valores acima de 0.85 em aplicações como radiologia e segmentação de tecidos em imagens médicas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

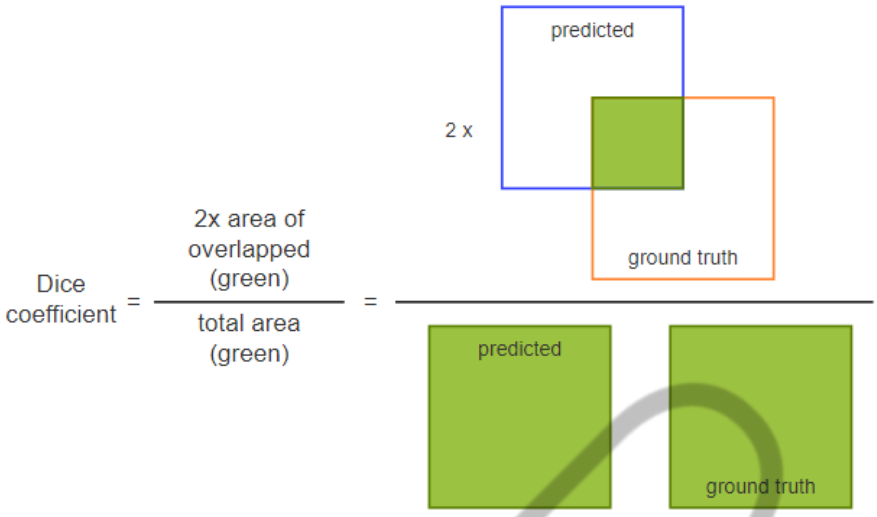


Figura 7 - Interpretação gráfica do Dice Coefficient
Fonte: Medium (2023), adaptado por FIAP (2025)

A **Pixel Accuracy mede a proporção de pixels corretamente classificados em toda a imagem**. Embora seja uma métrica simples, **pode ser enganosa em cenários onde há desequilíbrio entre classes**. Por exemplo, se a maioria da imagem pertence ao fundo e poucas áreas correspondem ao objeto de interesse, um modelo pode obter uma alta Pixel Accuracy simplesmente classificando a maioria dos pixels como fundo. Assim, um valor alto de Pixel Accuracy (acima de 90%) pode não indicar, necessariamente, uma boa segmentação caso o dataset seja desbalanceado.

Para resolver essa limitação, utiliza-se a **Mean Pixel Accuracy (mPA)**, que **calcula a precisão média de cada classe separadamente e depois tira a média geral**. Essa métrica é mais robusta para datasets onde diferentes classes têm distribuições desiguais, garantindo que objetos menores ou menos frequentes sejam considerados na avaliação do modelo.

Os modelos de segmentação podem ser avaliados por diferentes limiares, e métricas como **IoU@50** e **IoU@75** indicam a qualidade da segmentação considerando diferentes graus de precisão. Em aplicações práticas, segmentações gerais podem atingir IoU@50 entre 0.5 e 0.7, enquanto segmentações de alto nível, como em diagnósticos médicos e inspeções industriais, costumam apresentar IoU@75 superiores a 0.8.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CRIAÇÃO DE LABELS PARA DETECÇÃO E SEGMENTAÇÃO

Por serem tarefas de aprendizado supervisionado, a existência de labels bem construídos é essencial para viabilizar o treinamento e avaliação dos modelos de detecção e segmentação de objetos. Como aprendemos anteriormente, estes labels podem ser *bounding boxes* ou máscaras. Mas como estes labels são gerados e armazenados? É aqui que entra a **anotação de imagens**.

A anotação de imagens consiste em delimitar objetos e definir suas classes, criando conjuntos de dados que permitem que redes neurais aprendam a reconhecer e segmentar elementos com precisão. O uso de ferramentas especializadas agiliza e padroniza o processo, garantindo um conjunto de dados consistente e de alta qualidade.

Existem diversas ferramentas gratuitas que auxiliam nesse processo. O **LabelImg** é uma das opções mais populares para anotação de caixas delimitadoras, gerando arquivos no formato PASCAL VOC ou YOLO, amplamente utilizados em redes como YOLOv5 e Faster R-CNN. Já para segmentação de instâncias e semântica, o **CVAT** (Computer Vision Annotation Tool) se destaca, pois permite a anotação detalhada de imagens e vídeos com suporte para diferentes formatos de saída. Outra opção robusta é o **LabelMe**, que facilita a criação de anotações poligonais e pode ser usado diretamente em um navegador.

Outras ferramentas avançadas incluem o **Roboflow Annotate**, que oferece uma interface intuitiva e integração com plataformas de treinamento, e o **SuperAnnotate**, que combina anotação manual com automação baseada em IA para acelerar o processo. Para projetos colaborativos, o **Makesense.ai** permite a anotação diretamente no navegador sem necessidade de instalação.

Essas ferramentas simplificam o trabalho de anotação e garantem dados de qualidade, tornando o treinamento de modelos mais eficiente e acessível. A saída pode ser a posição, tamanho e categoria dos *bounding boxes* ou uma imagem de contorno do objeto desejado.

Nota: aqui nosso intuito não é ensinar a utilizar cada uma destas ferramentas, mas orientar sobre sua existência e importância!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É importante lembrar também que, por ser uma atividade subjetiva, a qualidade da anotação depende da precisão dada pelo anotador, ou seja, podemos listar uma série de desafios da anotação de imagens:

- **Alto custo de tempo e mão de obra:** a anotação manual é um processo demorado, especialmente para segmentação pixel a pixel.
- **Inconsistência entre anotadores:** diferenças na interpretação podem levar a anotações imprecisas, reduzindo a qualidade dos dados.
- **Desafios em objetos sobrepostos:** separar corretamente objetos que estão muito próximos ou se sobrepõem pode ser complexo.
- **Necessidade de formatos adequados:** diferentes modelos exigem anotações em formatos específicos (YOLO, COCO, PASCAL VOC etc.).
- **Uso de dados ruidosos ou irrelevantes:** imagens mal selecionadas podem comprometer o desempenho do modelo.
- **Limitações de automação:** ferramentas de anotação assistida ainda requerem validação humana para garantir precisão.
- **Privacidade e regulamentação:** em setores como saúde e segurança, a anotação de imagens pode esbarrar em restrições legais e de proteção de dados.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 MÉTODOS CLÁSSICOS

Antes do avanço das redes neurais profundas, a detecção e segmentação de objetos eram resolvidas por meio de métodos clássicos baseados em características manuais. Esses métodos consistiam em extrair manualmente padrões visuais, como bordas, texturas e formas geométricas, e utilizar algoritmos tradicionais para identificar objetos em imagens. Durante décadas, técnicas como Haar Cascades, HOG (Histogram of Oriented Gradients) e SIFT (Scale-Invariant Feature Transform) foram amplamente utilizadas para reconhecimento facial, rastreamento de objetos e análise de imagens médicas.

Tais métodos clássicos exigiam engenharia de features detalhada e dependiam fortemente da qualidade da iluminação, escala e ângulo da imagem, tornando sua aplicação menos robusta em cenários complexos. No entanto, elas desempenharam um papel fundamental no avanço da visão computacional, sendo a base para o desenvolvimento dos modelos modernos de Deep Learning aplicados a detecção e segmentação.

4.1 Algoritmo de Haar Cascades

O **Haar Cascades** é um algoritmo clássico de detecção de objetos baseado em características manuais, amplamente utilizado em reconhecimento facial e outras aplicações de visão computacional. Desenvolvido por Paul Viola e Michael Jones em 2001, esse método utiliza cascatas de classificadores treinados para detectar padrões visuais em imagens de forma rápida e eficiente (os padrões estão ilustrados na figura “Padrões típicos buscados pelo algoritmo de Haar Cascades”). Seu principal diferencial é o uso de janelas deslizantes que percorrem a imagem analisando diferentes regiões para encontrar objetos de interesse, como rostos ou placas de trânsito.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

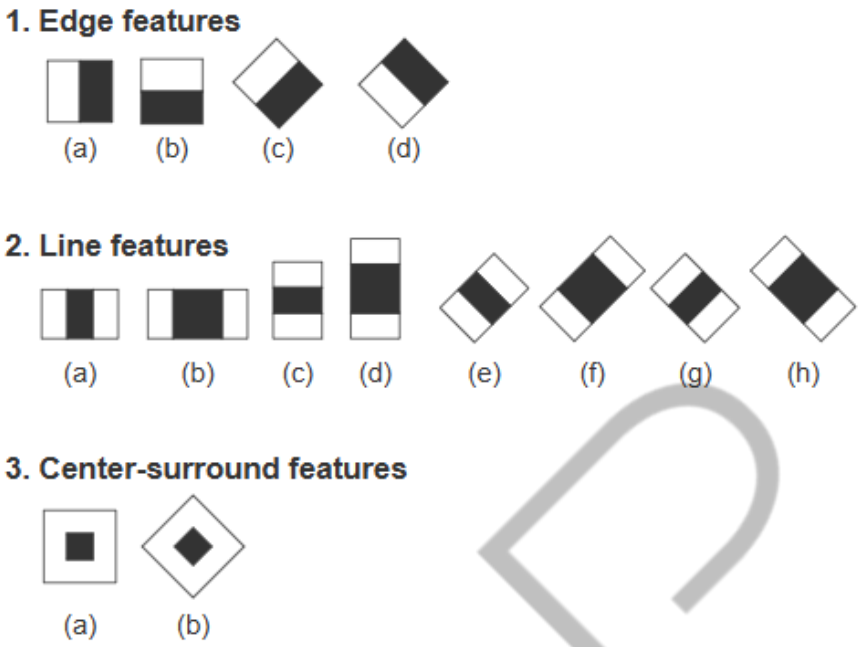


Figura 8 - Padrões típicos buscados pelo algoritmo de Haar Cascades
Fonte: Starck Overflow (2014), adaptado por FIAP (2025)

4.1.1 Como o Haar Cascades funciona?

O algoritmo opera em quatro etapas principais:

1. **Extração de características com Haar Features:**

- As Haar Features são padrões visuais extraídos da imagem, baseados na diferença de intensidade entre regiões claras e escuras.
- Exemplos comuns incluem bordas, linhas horizontais e verticais e regiões em forma de quadrado, similares a **filtros que detectam contornos e texturas**.

2. **Aplicação do Integral Image para otimização:**

- Para tornar a extração de características mais rápida, o algoritmo utiliza a Integral Image, uma estrutura matemática que **permite calcular rapidamente somas de pixels em regiões específicas**.
- Isso torna a análise de grandes imagens mais eficiente, **reduzindo o custo computacional**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3. **Treinamento do classificador usando Adaboost:**

- Durante o treinamento o algoritmo utiliza o Adaboost, um método de Aprendizado de Máquina que **seleciona as características mais relevantes para a detecção**.
- O classificador é treinado com imagens positivas (contendo o objeto) e imagens negativas (sem o objeto), ajustando pesos para melhorar a precisão.

4. **Uso da cascata de classificadores:**

- Em vez de analisar toda a imagem de uma vez, o Haar Cascades emprega uma cascata de classificadores, onde cada estágio descarta rapidamente regiões improváveis de conter o objeto.
- Apenas as regiões que passam por todas as etapas da cascata são consideradas como detecções válidas, tornando o algoritmo rápido e eficiente.

4.1.2 Aplicações e limitações

O Haar Cascades foi muito utilizado em reconhecimento facial em tempo real, sendo integrado em frameworks como OpenCV. No entanto, tem algumas limitações:

- **Vantagens:** rápido, eficiente e adequado para dispositivos com baixo poder computacional.
- **Desvantagens:** sensível a variações de iluminação, rotação e escala, além de ter uma taxa de falsos positivos maior em comparação com redes neurais modernas.

Apesar dessas limitações, o Haar Cascades foi um marco na visão computacional e serviu de base para os métodos mais avançados de detecção baseados em Deep Learning.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 Algoritmo HOG + SVM

O HOG + SVM é um método clássico de detecção de objetos que combina Histogramas de Gradientes Orientados (HOG) para extração de características e um classificador de Máquina de Vetores de Suporte (SVM) para identificação dos objetos. Esse método foi amplamente utilizado em aplicações como detecção de pedestres e veículos, sendo um dos primeiros a alcançar bons resultados em imagens reais antes do surgimento das redes neurais profundas. Observe a figura:

Figura 9 - Exemplo de descritor de features HOG na imagem
Fonte: Medium (2020)

4.2.1 Como o HOG + SVM funciona?

O algoritmo funciona em duas etapas principais:

1. **Extração de Características com HOG:**

- O HOG transforma a imagem em um mapa de gradientes, analisando como os pixels mudam de intensidade em diferentes direções.
- A imagem é dividida em pequenas regiões chamadas células, e dentro de cada célula, os gradientes são computados para capturar a estrutura do objeto.
- A ideia é que a distribuição dessas direções de gradiente represente padrões visuais robustos, como bordas e texturas, independente de iluminação ou cor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2. **Classificação com SVM:**

- Após a extração dos descritores HOG, o vetor de características é alimentado em um classificador SVM treinado previamente.
- O SVM separa as classes (por exemplo, “pedestre” ou “não pedestre”) através de um hiperplano de decisão, maximizando a margem entre os exemplos positivos e negativos.
- Esse classificador é treinado em um grande conjunto de imagens anotadas, garantindo que o modelo consiga diferenciar padrões reais.

4.2.2 Aplicações e limitações

- **Vantagens:**
  - Robustez contra variações de iluminação e pequenas mudanças no objeto.
  - Menos sensível a ruídos do que Haar Cascades.
  - Funciona bem para detectar padrões estruturados, como corpos humanos ou placas de trânsito.
- **Desvantagens:**
  - Computacionalmente mais pesado que Haar Cascades, pois exige mais cálculos na fase de extração de características.
  - Menos eficiente em cenários complexos, com grande variação de poses e ângulos.
  - Não realiza aprendizado profundo, ou seja, não consegue aprender características complexas como as redes neurais.

Apesar dessas limitações, HOG + SVM foi um grande avanço para a detecção de objetos e serviu como base para técnicas modernas, como redes neurais convolucionais (CNNs), que substituíram esse método em diversas aplicações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.3 Limitações dos métodos tradicionais

Apesar de desempenharem um papel significativo por muito tempo para detectar e segmentar objetos, os métodos tradicionais listados acima enfrentaram diversos obstáculos à medida que as aplicações se tornavam mais complexas.

Entre as limitações mais comuns, destacam-se:

- **Dependência da engenharia de features manual**, demandando conhecimento especializado, além de ser limitado em generalização, pois os modelos não se adaptavam facilmente a novos contextos.
- **Sensibilidade a iluminação, escala e posição**, sendo que pequenas alterações no ambiente de produção da imagem poderiam comprometer a detecção, tornando o modelo pouco confiável para situações reais.
- **Dificuldade em identificar múltiplos objetos na cena**, necessitando varrer toda a imagem com janelas deslizantes, tornando a abordagem computacionalmente cara e ineficiente.
- **Segmentação ineficiente**, sendo feita via classificação pixel a pixel entre “objeto” ou “background”. Além disso, havia baixa precisão frente a imagens complexas ou ruidosas.

Com essas limitações, tornou-se evidente que era necessário um modelo mais adaptável, eficiente e robusto para detecção e segmentação de objetos. O Deep Learning surgiu como uma alternativa aos métodos tradicionais de classificação, revolucionando a área de pesquisa ao eliminar a necessidade de engenharia manual de features e permitir que as redes neurais aprendessem automaticamente padrões complexos nos dados.

A figura a seguir mostra, de maneira não exaustiva, as principais Redes Neurais Profundas para aplicações em processamento de imagens:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

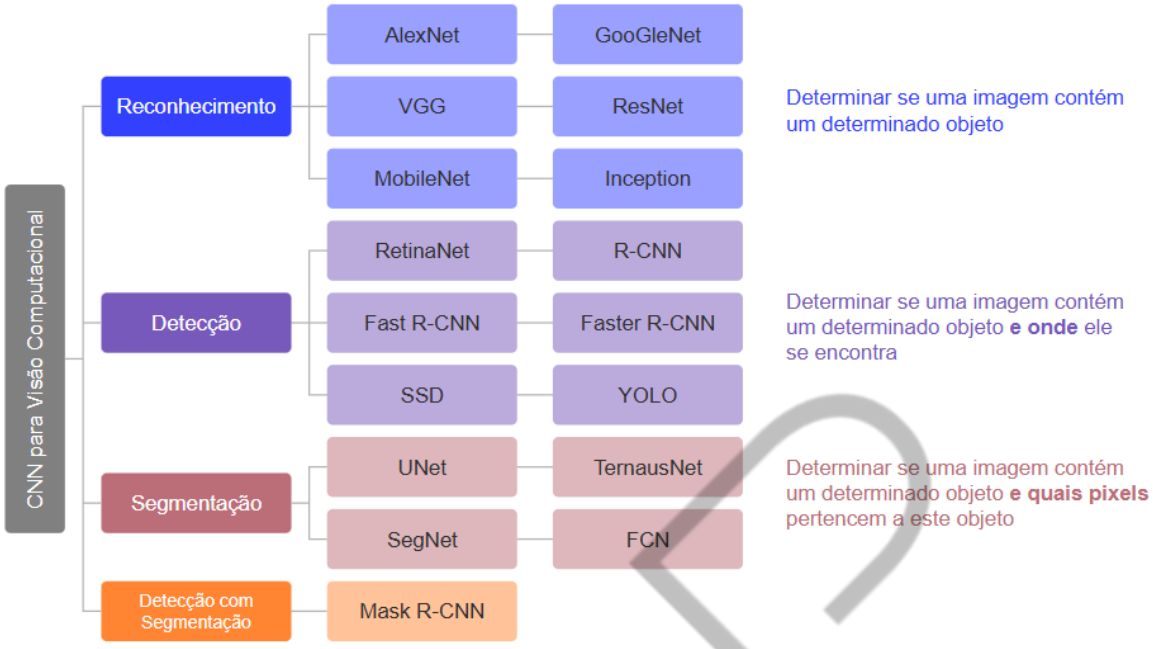


Figura 10 - Principais arquiteturas de Deep Learning para processamento de imagens
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

A evolução das redes neurais profundas permitiu que sistemas de visão computacional alcançassem um nível de precisão e eficiência antes impossível, tornando aplicações como veículos autônomos, reconhecimento facial e diagnóstico médico uma realidade viável e confiável.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 REDES NEURAIS PARA DETECÇÃO DE OBJETOS

A detecção de objetos baseada em redes neurais profundas revolucionou a visão computacional ao substituir métodos tradicionais por abordagens que aprendem diretamente com os dados, tornando a identificação e localização de objetos mais precisa e eficiente. Diferentes arquiteturas surgiram ao longo do tempo, aprimorando o processo de detecção. Entre elas, destacam-se as abordagens baseadas na família R-CNN (Region-based Convolutional Neural Networks) e os modelos da linha YOLO (You Only Look Once), que trouxeram avanços significativos em velocidade e desempenho.

5.1 Breve histórico

A R-CNN foi uma das primeiras arquiteturas a aplicar redes neurais convolucionais na detecção de objetos. A próxima figura ilustra seu funcionamento:

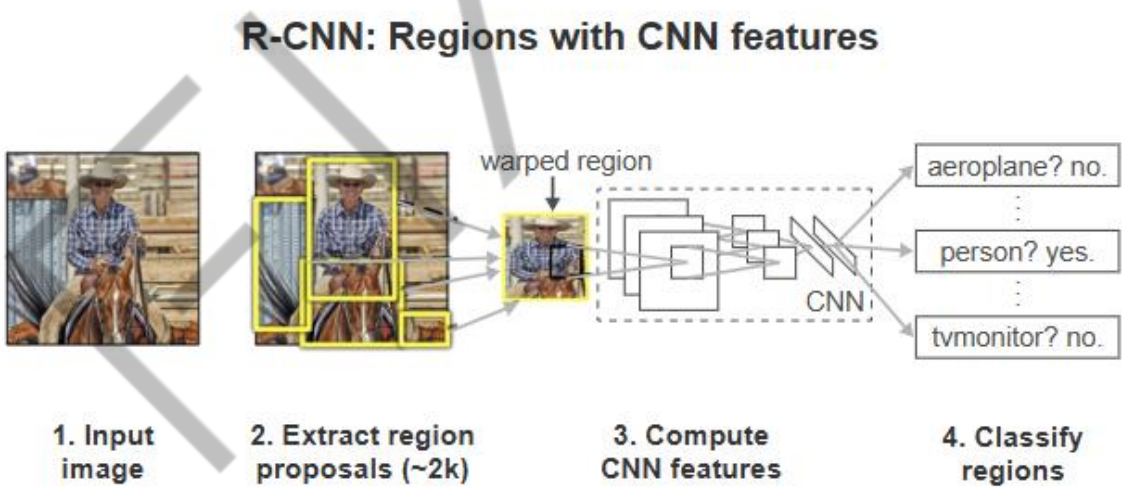


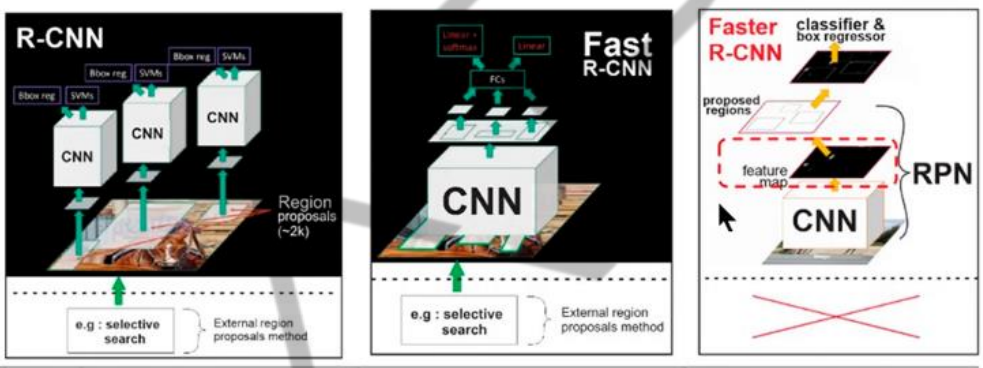
Figura 11 - Arquitetura R-CNN
Fonte: Medium (2024), adaptado por FIAP (2025)

O primeiro modelo gera regiões de interesse (propostas de objetos) em uma imagem e, em seguida, aplica uma CNN separadamente para cada região. Embora tenha melhorado a precisão da detecção em comparação com abordagens anteriores, a R-CNN era lenta, pois cada imagem precisava passar por múltiplas inferências para processar todas as regiões candidatas. Para contornar essa limitação, foi

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

desenvolvida a Fast R-CNN, que aprimorou a eficiência ao aplicar a CNN apenas uma vez sobre a imagem inteira, extraindo um mapa de características antes de processar as regiões de interesse.

Mesmo assim, ainda havia um gargalo computacional na etapa de geração de regiões propostas. A solução veio com a Faster R-CNN, que introduziu a Rede de Propostas de Região (RPN — Region Proposal Network), eliminando a necessidade de um método externo para sugerir regiões e permitindo que o modelo aprendesse a propor objetos de maneira integrada. O processo está ilustrado na figura a seguir, assim como uma breve comparação da eficiência dos modelos.



	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image	50 seconds	2 seconds	0.2 seconds
Speed-up	1x	25x	250x
mAP (VOC 2007)	66.0%	66.9%	66.9%

Figura 12 - Comparação das arquiteturas R-CNN, Fast R-CNN e Faster R-CNN
Fonte: YouTube (2017)

5.2 Arquitetura YOLO

Apesar dessas melhorias, as arquiteturas R-CNN ainda tinham um problema crítico: a detecção era realizada em duas etapas — primeiro a proposta de regiões e depois a classificação, o que impactava a velocidade. Foi nesse contexto que surgiu a YOLO (You Only Look Once), um modelo que transformou completamente a detecção de objetos ao adotar uma abordagem one-shot, ou seja, processando a imagem inteira de uma só vez e prevendo as localizações e categorias dos objetos simultaneamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Diferente da Faster R-CNN, que examina várias regiões separadamente, a YOLO divide a imagem em uma grade e, para cada célula, prevê diretamente caixas delimitadoras (*bounding boxes*), classes e probabilidades associadas. Analise a figura:

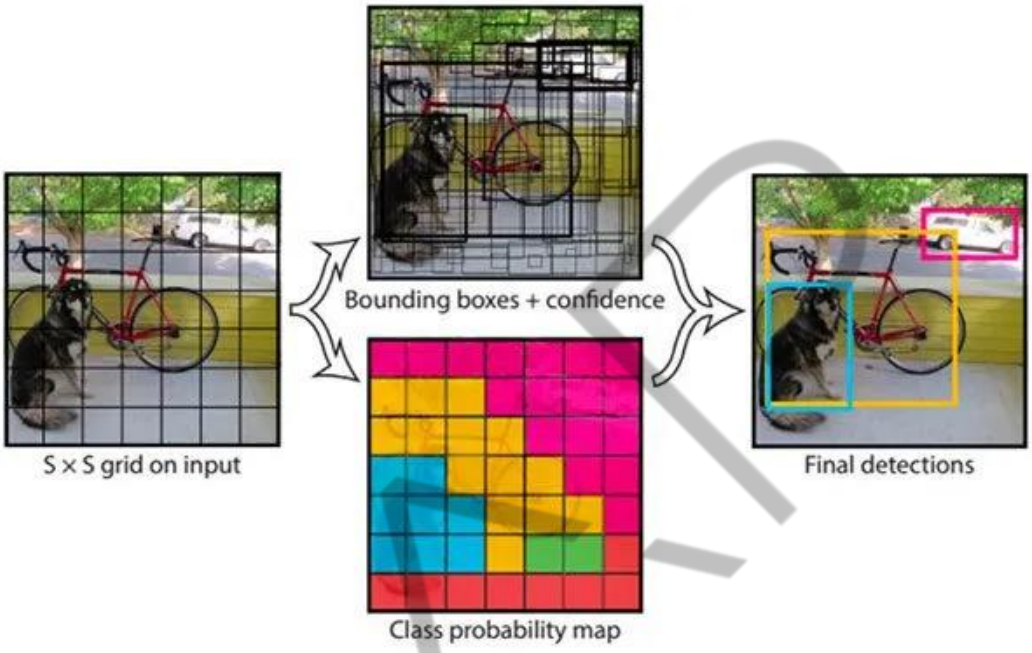


Figura 13 - Intuição da arquitetura YOLO
Fonte: SuperAnnotate (2023)

A principal vantagem da YOLO é sua velocidade extremamente alta, tornando-a ideal para aplicações que exigem processamento em tempo real, como veículos autônomos, sistemas de segurança e análise de vídeos ao vivo.

Além da rapidez, a YOLO apresenta uma excelente capacidade de generalização, permitindo detectar objetos em diferentes cenários, mesmo com variações de iluminação e perspectiva.

Outra característica notável é que, ao contrário dos modelos R-CNN que analisam regiões separadamente, a YOLO considera o contexto global ao realizar a detecção, o que reduz falsos positivos e melhora a precisão da predição.

5.2.1 Detalhamento

Como mencionado, o diferencial da YOLO é passar pelas imagens uma única fez, detectando objetos e reconhecendo-os. As etapas cruciais deste processo são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Divisão da imagem e extração de características:** a YOLO processa a imagem por meio de uma CNN e a divide em uma SxS. Cada célula dessa grade é responsável por prever objetos que tenham seu centro dentro dela. Por exemplo, se uma imagem for dividida em uma grade de 7 × 7, teremos 49 células, e cada uma delas tentará prever objetos presentes na sua região. Durante essa etapa, a rede extrai características visuais, como texturas, bordas e padrões. Essa extração ocorre por meio de camadas convolucionais seguidas de camadas de pooling, que reduzem a dimensionalidade da imagem, mantendo as informações essenciais para a detecção.
- **Previsão de caixas delimitadoras (*bounding boxes*):** cada célula da grade não apenas identifica se há um objeto ou não, mas também prevê múltiplas caixas delimitadoras (*bounding boxes*). Para cada caixa prevista, a YOLO estima:
  - **As coordenadas x,y,w,h (x, y, width e height):** representam o centro da caixa e suas dimensões (largura e altura) em relação à célula da grade.
  - **A confiança da detecção:** indica a probabilidade de a caixa conter um objeto real, baseada na sobreposição com o objeto verdadeiro (IoU — Intersection over Union).
  - **A classe do objeto:** a rede também prevê qual categoria o objeto pertence, por meio de probabilidades associadas a cada classe treinada.

Observação: cada célula pode prever várias caixas, mas apenas aquelas com alta confiança são mantidas.

- **Supressão de previsões duplicadas:** para evitar múltiplas detecções redundantes de um mesmo objeto, a YOLO utiliza a técnica de Non-Maximum Suppression (NMS). O NMS filtra as caixas sobrepostas, mantendo apenas a caixa com a maior confiança para cada objeto, eliminando previsões duplicadas e melhorando a precisão da detecção.

Ao longo das versões, a YOLO foi aprimorada, passando por variações como YOLOv3, YOLOv4 e YOLOv5, cada uma trazendo refinamentos na precisão e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

eficiência. Mais recentemente, arquiteturas como YOLOv7 e YOLOv8 continuaram a evoluir, tornando-se ainda mais rápidas e robustas. Hoje, a YOLO é amplamente utilizada em diversas aplicações, desde a detecção de objetos em drones e veículos autônomos até análises em esportes e sistemas biométricos. Sua combinação de desempenho, eficiência e precisão faz dela uma das arquiteturas mais populares e eficazes para detecção de objetos em tempo real.

5.2.2 Evolução em suas versões

Desde sua criação, a YOLO passou por várias versões que aprimoraram sua eficiência:

- **YOLOv1 (2015):** introduziu o conceito de grade SxS para previsões únicas.
- **YOLOv2 e YOLOv3:** melhoraram a precisão ao introduzir camadas mais profundas e técnicas como Anchor Boxes. Estas são um conjunto de caixas predefinidas com diferentes proporções que são usadas para detectar objetos de diferentes formatos.
- **YOLOv4 e YOLOv5:** otimizaram a arquitetura, tornando-a ainda mais rápida e eficiente.
- **YOLOv7 e YOLOv8:** trouxeram melhorias na robustez da detecção, reduzindo erros em objetos pequenos e sobrepostos.

Versões subsequentes do YOLO (v8 em diante) foram desenvolvidas por diferentes pesquisadores, melhorando ainda mais o desempenho e introduzindo novos recursos. Essas versões não estão oficialmente associadas aos autores originais do YOLO, mas baseiam-se em seu trabalho. A partir de 2024, existem versões até YOLO11.

As evoluções melhoraram significativamente a precisão da rede. A figura a seguir mostra a diferença entre a YOLOv3 e a v4 em termos de Average Precision (AP). Note a melhoria significativa da v4, mesmo em diversas configurações de frames por segundo. A “MS COCO” é uma base de dados referência para avaliação de tarefas de detecção de objetos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

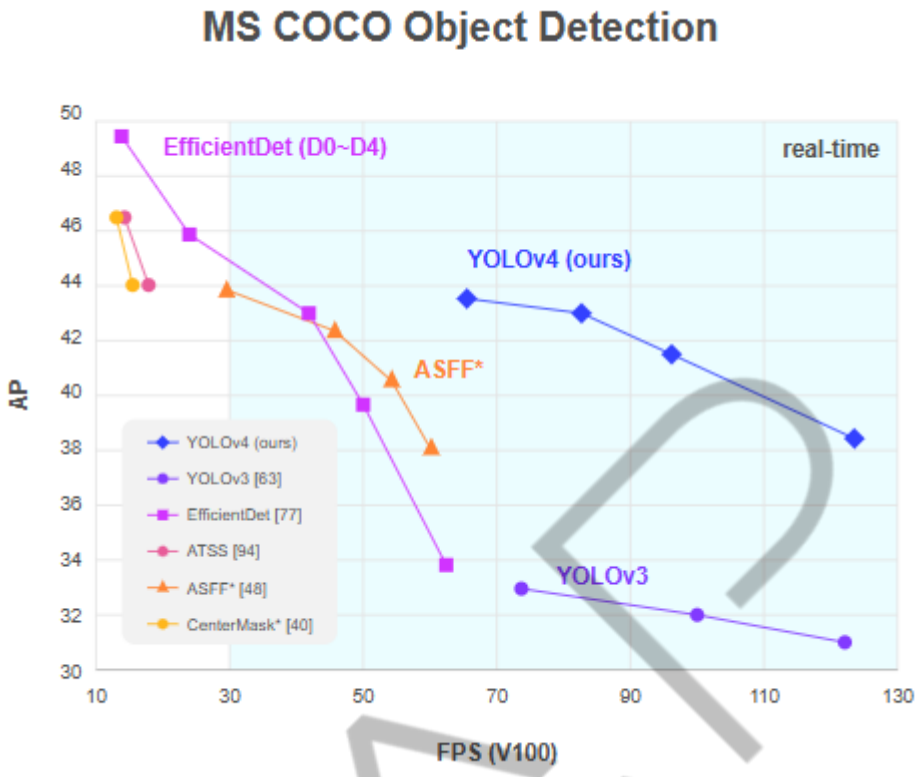


Figura 14 - Comparação da YOLOv3 com a YOLOv4 na base MS COCO
Fonte: arxiv (2020), adaptado por FIAP (2025)

A YOLO continua sendo um dos modelos mais utilizados para detecção de objetos, combinando velocidade e precisão para aplicações do mundo real.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 REDES NEURAIS PARA SEGMENTAÇÃO DE OBJETOS

6.1 Tipos de segmentação

Antes de prosseguirmos, é importante distinguir duas formas de segmentação: a **segmentação semântica** e a **segmentação de instâncias**. Observe a figura:

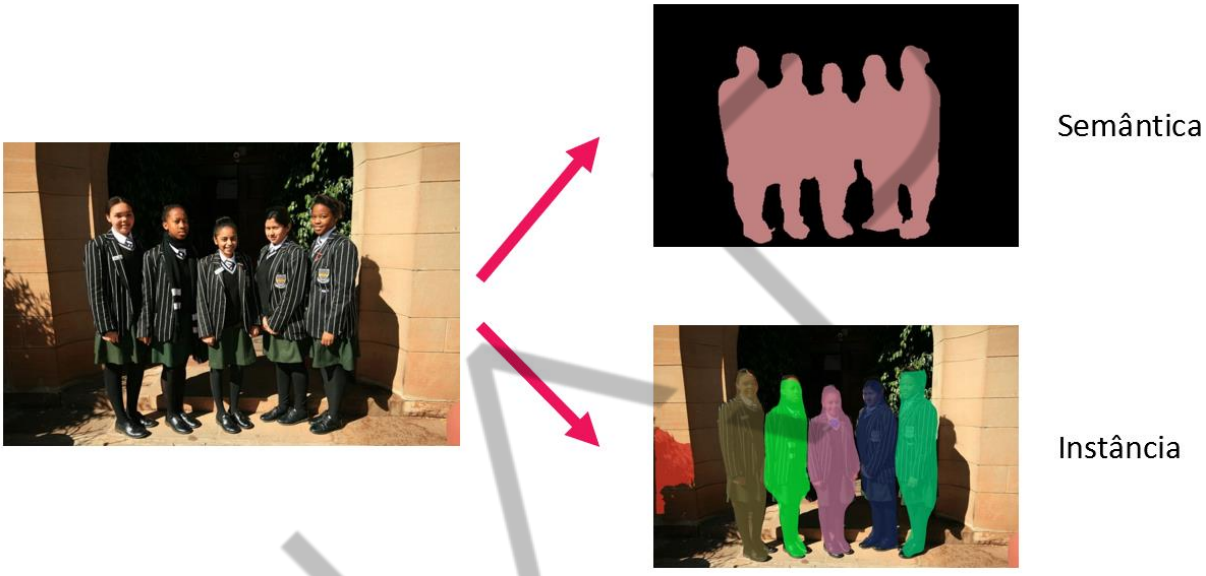


Figura 15 - Tipos de segmentação
Fonte: Elaborado pelo autor (2015)

A **segmentação semântica** atribui um rótulo a cada pixel da imagem, classificando todas as áreas pertencentes a uma mesma categoria sem diferenciar instâncias individuais. Por exemplo, em uma cena com múltiplos carros, todos os pixels pertencentes a veículos serão marcados como “carro”, sem distinguir entre um e outro. Esse método é amplamente utilizado em aplicações como análise médica, onde é necessário segmentar órgãos ou tecidos, e em veículos autônomos, para diferenciar áreas como estrada, calçada e pedestres.

Já a **segmentação de instância** vai além, pois não apenas classifica os pixels, mas também diferencia cada objeto individualmente, mesmo que pertençam à mesma classe. No caso de uma imagem com vários carros, a segmentação de instância identificaria cada um separadamente, permitindo um rastreamento preciso. Essa abordagem é essencial para aplicações como detecção de múltiplos objetos em vídeos, reconhecimento facial e realidade aumentada, onde a diferenciação entre

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

instâncias é crucial para interações precisas. Modelos como Mask R-CNN são amplamente utilizados para essa tarefa, tornando a segmentação mais detalhada e funcional para diversos cenários.

6.2 Breve histórico

A segmentação de objetos é uma das tarefas mais complexas e essenciais na visão computacional, pois permite que algoritmos identifiquem precisamente cada objeto em nível de pixel. Diferente da detecção, que apenas localiza objetos por meio de caixas delimitadoras, a segmentação é fundamental em áreas como medicina, automação industrial, veículos autônomos e realidade aumentada, onde a precisão na forma dos objetos é crucial. Com a evolução do Deep Learning, surgiram arquiteturas poderosas para resolver esse problema, como U-Net, SegNet e Fully Convolutional Networks (FCN), que revolucionaram o campo ao utilizar redes neurais convolucionais para realizar segmentação de forma mais eficiente e precisa.

A U-Net, desenvolvida inicialmente para segmentação biomédica, tornou-se uma das arquiteturas mais populares devido à sua capacidade de segmentar objetos mesmo com poucos dados. Seu formato em “U” se deve à sua estrutura simétrica, onde um **caminho de contração** (encoder) reduz a dimensionalidade da imagem para extrair características importantes e um **caminho de expansão** (decoder) reconstrói a segmentação preservando detalhes. Esse modelo é amplamente utilizado em análises médicas, mapeamento geoespacial e para processar imagens satelitais. Observe a figura:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

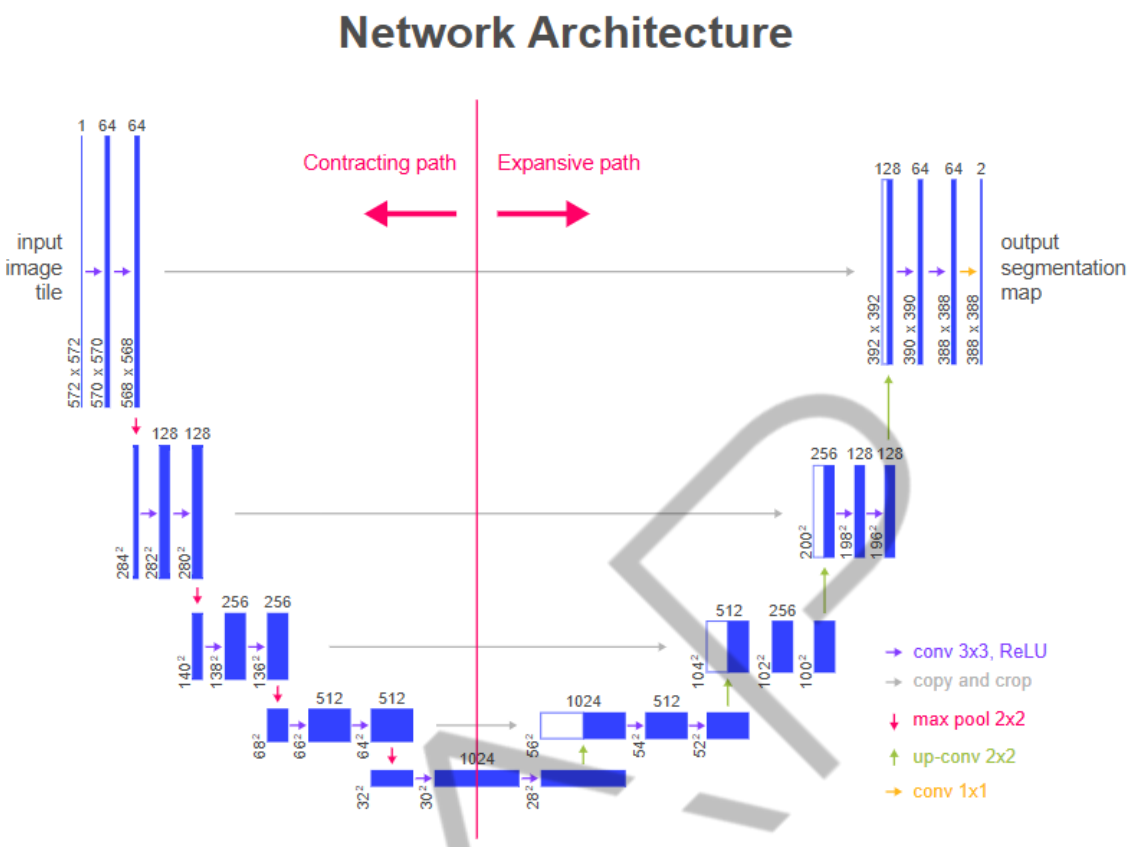


Figura 16 - Arquitetura U-Net
Fonte: Medium (2019), adaptado por FIAP (2025)

Já a SegNet, uma variação baseada em codificadores e decodificadores, se destaca por ser mais eficiente em termos de memória, sendo muito utilizada em segmentação semântica de cenas urbanas, como no reconhecimento de ruas e calçadas para carros autônomos. Por outro lado, a Fully Convolutional Network (FCN) foi uma das primeiras arquiteturas a eliminar camadas densas na segmentação, permitindo que todo o processo fosse feito apenas com convoluções, tornando o modelo mais eficiente e robusto.

6.3 Arquitetura SAM

Embora essas arquiteturas tenham impulsionado o avanço da segmentação de objetos, elas ainda possuem desafios, como a necessidade de ajustes manuais para diferentes domínios e treinamento específico para cada aplicação. Para superar essas limitações, a Meta AI desenvolveu o Segment Anything Model (SAM), um modelo inovador que promete segmentação universal, sendo capaz de segmentar qualquer

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

objeto em qualquer imagem ou vídeo sem necessidade de um treinamento específico. O SAM introduz uma abordagem prompt-based, onde os usuários podem fornecer diferentes tipos de instruções para guiar a segmentação, como seleção de pontos, caixas delimitadoras ou até mesmo comandos textuais. Observe a figura “Arquitetura SAM”.

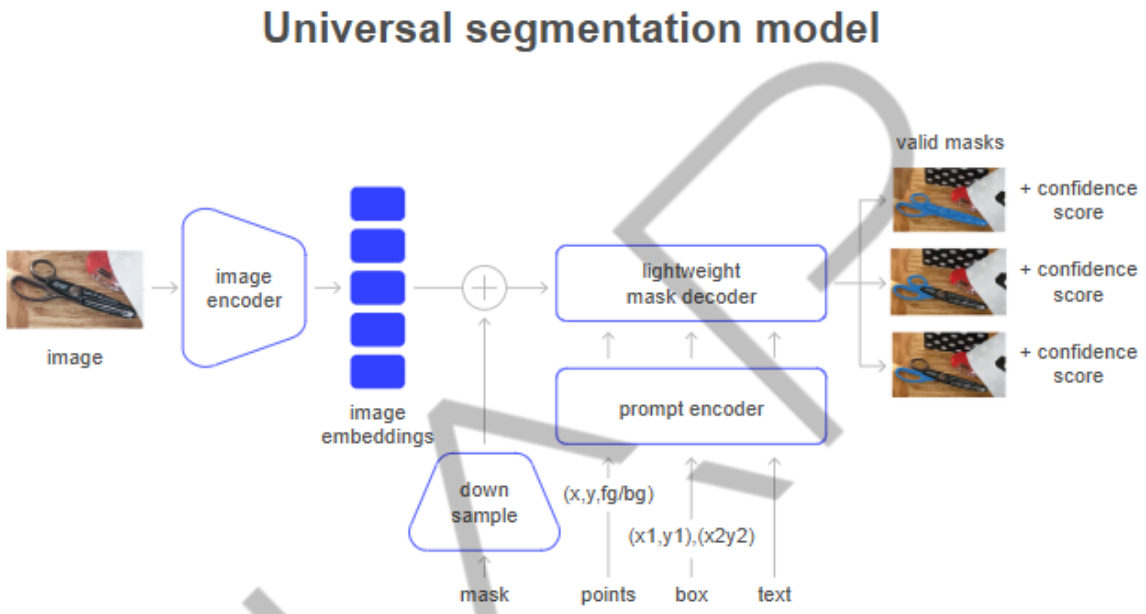


Figura 17 - Arquitetura SAM
Fonte: Medium (2023), adaptado por FIAP (2025)

O diferencial do SAM está na sua generalização sem precedentes. Enquanto modelos tradicionais precisam ser treinados para um conjunto específico de objetos e cenários, o SAM foi pré-treinado em um enorme volume de dados, tornando-se capaz de segmentar qualquer objeto, mesmo aqueles que nunca viu antes. Essa característica abre portas para aplicações extremamente diversas, desde edição automática de imagens e vídeos, até usos na indústria, biomedicina e robótica. Além disso, o SAM permite interação direta do usuário, possibilitando refinamentos e ajustes personalizados na segmentação em tempo real, algo que os modelos tradicionais não oferecem com facilidade.

Com essa evolução, a segmentação de objetos está se tornando cada vez mais acessível, poderosa e adaptável a diferentes cenários. Modelos como SAM representam um grande avanço, permitindo que qualquer usuário,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

independentemente do seu conhecimento técnico, possa utilizar segmentação de alta precisão para diversas finalidades.

6.3.1 Detalhamento

O SAM representa um avanço significativo na segmentação de objetos, trazendo uma abordagem inovadora baseada em generalização e interatividade. Diferente dos modelos tradicionais, que exigem treinamento específico para cada aplicação, o SAM é um modelo pré-treinado capaz de segmentar qualquer objeto em qualquer imagem sem necessidade de ajustes adicionais. Isso o torna uma solução altamente flexível para tarefas de visão computacional, realidade aumentada, edição de imagens e inteligência artificial interativa.

Seus principais blocos de funcionamento estão descritos a seguir:

- **Arquitetura e extração de características:** o SAM utiliza uma rede neural convolucional combinada com um Transformer para processar imagens e identificar padrões visuais complexos. Sua arquitetura é composta por três módulos principais:
  - **Encoders de imagem:** extraem características visuais a partir da imagem de entrada, criando um mapa de características detalhadas que será usado para segmentação.
  - **Mecanismo de prompts (interação do usuário):** permite que a segmentação seja refinada com diferentes entradas, como pontos, caixas delimitadoras ou descrições textuais.
  - **Decoder de máscaras:** a partir das informações extraídas pelo encoder e dos prompts do usuário, o SAM gera máscaras de segmentação altamente precisas, separando objetos em nível de pixel.
- **Interatividade e generalização:** uma das grandes inovações do SAM é sua capacidade interativa. Diferente dos modelos tradicionais, que segmentam objetos de forma automática, o SAM pode ser guiado pelo usuário para refinar e ajustar a segmentação. Isso significa que é possível indicar pontos de interesse, selecionar caixas ou até mesmo fornecer comandos textuais para definir quais objetos devem ser segmentados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além disso, o SAM foi treinado em um conjunto massivo de dados contendo bilhões de imagens, o que lhe permite generalizar bem mesmo para objetos que nunca viu antes. Essa generalização amplia suas aplicações, permitindo seu uso em setores como saúde, edição de imagens, robótica e segurança.

- **Supressão de previsões irrelevantes e ajuste dinâmico:** para evitar segmentações erradas ou imprecisas, o SAM utiliza um mecanismo de refinamento dinâmico, onde o usuário pode interagir com a saída da rede para ajustar a segmentação. Além disso, técnicas como atenção hierárquica e aprendizado auto-supervisionado garantem que o modelo refine sua saída conforme novas informações são fornecidas.

6.3.2 O impacto do SAM na visão computacional

O SAM inaugura uma nova era da segmentação de objetos, tornando-a mais acessível e intuitiva. Sua abordagem interativa e generalizável pode substituir a necessidade de modelos específicos para cada aplicação, facilitando a adoção da segmentação em novos setores.

O poder de segmentação do SAM é inegável. A próxima figura ilustra este fenômeno: temos a imagem original e sua segmentação. Cada camada colorida representa um “objeto” contínuo reconhecido pelo SAM. O nível de precisão é tanto que diferencia partes do gramado, peças de roupa e rostos distantes na multidão.

Figura 18 - Exemplo de segmentação com SAM
Fonte: Segment Anything (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O nível de detecção pode ser parametrizado no uso da ferramenta, a qual está disponível de forma gratuita. Mais uma vitória da comunidade Open Source.

À medida que modelos baseados em prompts ganham espaço, é provável que a segmentação de imagens se torne cada vez mais integrada a sistemas autônomos, IA embarcada e interfaces inteligentes, revolucionando a maneira como máquinas compreendem e processam imagens.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 HANDS ON: DETECTANDO E SEGMENTANDO OBJETOS

Agora que conhecemos estratégias modernas de detecção e segmentação de objetos via Deep Learning, está na hora de praticarmos sua implementação. Nesta atividade não treinaremos redes profundas do zero, mas iremos interagir com modelos pré-treinados. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 3), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para estes códigos, utilizaremos imagens já salvas nos repositórios dos modelos. Se preferir, você poderá testá-los com imagens próprias!

7.1 Detecção de objetos com YOLO

Vamos começar as instalações e setup do ambiente:

```
!git clone https://github.com/pjreddie/darknet
%cd darknet
!make
!wget https://pjreddie.com/media/files/yolov3.weights
```

Código-fonte 1 - Instalações necessárias para YOLO
Fonte: Elaborado pelo autor (2025)

A célula anterior indica os passos de instalação e download de modelos pré-treinados. Por ser extensa, vamos omitir a saída. Podemos avançar para os imports, e a criação do método que exibe a figura com as predições da YOLO. O código a seguir não possui saída:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Imports
import numpy as np
import matplotlib.pyplot as plt
import cv2
import os
import time

%matplotlib inline

def predictImage():
  image = cv2.imread("/content/darknet/predictions.jpg")

  plt.imshow(cv2.cvtColor(image,
                          cv2.COLOR_BGR2RGB))
  plt.show()
```

Código-fonte 2 - Imports e método de exibição de resultados
Fonte: Elaborado pelo autor (2025)

Agora podemos usar o modelo pré-treinado para detectar objetos usando a YOLO-v3 pré-treinada para extrair informações de imagens passadas como input. A chamada é feita evocando uma linha de comando, e então exibindo a figura resultante.

```
!./darknet detect cfg/yolov3.cfg yolov3.weights data/dog.jpg
predictImage()
```

Código-fonte 3 - Usando a YOLO para detectar objetos e exibindo os resultados
Fonte: Elaborado pelo autor (2025)

O resultado está ilustrado na próxima figura. Você pode experimentar outras saídas, substituindo a última parte do comando (**trecho em negrito**) pela figura que deseja reconhecer objetos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

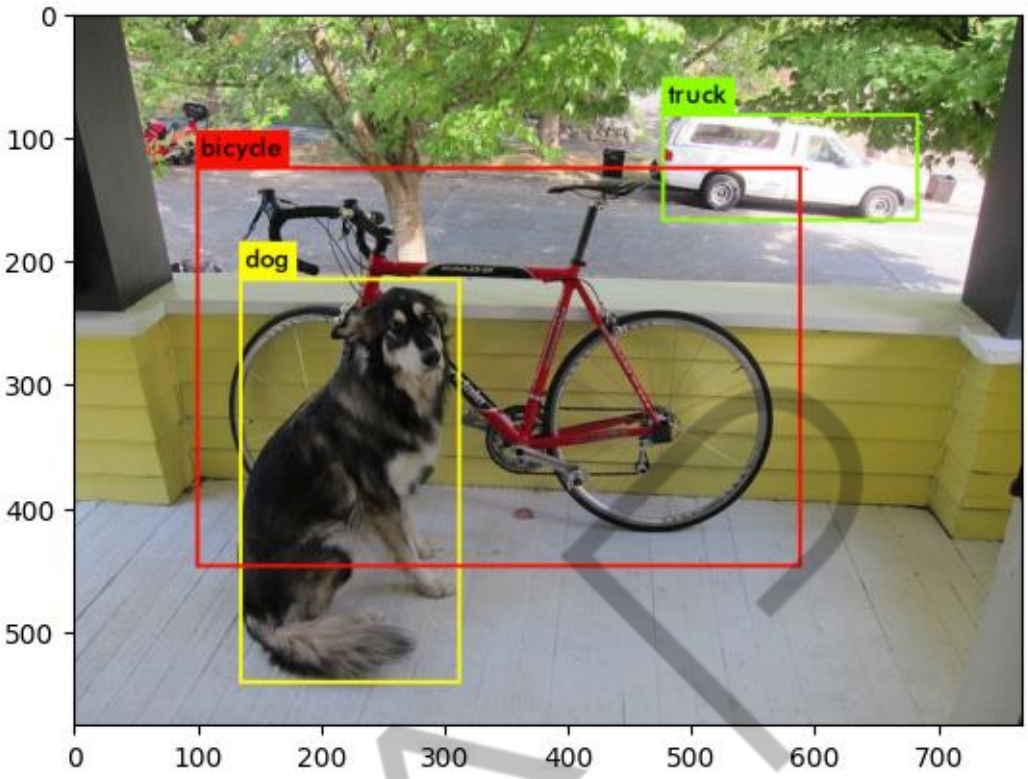


Figura 19 - Saída da execução do código-fonte 3
Fonte: Elaborado pelo autor (2025)

7.2 Segmentação de objetos com SAM

Vamos começar as instalações e setup do ambiente:

```
!pip install opencv-python matplotlib
!pip install 'git+https://github.com/facebookresearch/segment-anything.git'
!wget https://dl.fbaipublicfiles.com/segment_anything/sam_vit_h_4b8939.pth
```

Código-fonte 4 - Instalações necessárias para SAM
Fonte: Elaborado pelo autor (2025)

A célula anterior indica os passos de instalação e download de modelos pré-treinados. Por ser extensa, vamos omitir a saída. Podemos avançar para os imports e carregamento da imagem de entrada:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import numpy as np
import torch
import matplotlib.pyplot as plt
import cv2

import sys
sys.path.append("..")
from segment_anything import sam_model_registry, SamAutomaticMaskGenerator, SamPredictor

image = cv2.imread('pets.png')  # Altere aqui a imagem a ser segmentada
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10,10))
plt.imshow(image)
plt.axis('off')
plt.show()
```

Código-fonte 5 - Imports e carregamento da imagem de entrada
Fonte: Elaborado pelo autor (2025)

Ao executar a célula acima, você verá a figura "*pets.png*", usada como imput:

Figura 20 - Imagem de entrada para SAM
Fonte: Reproduzida do Google Images (2025)

Agora é a hora de chamar a biblioteca para realizar a segmentação da imagem de entrada. Existem diversas formas de configurar o gerador de máscaras das imagens, por exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     sam_checkpoint = "sam_vit_h_4b8939.pth"
     model_type = "vit_h"

     device = "cuda"

     sam                                                    =
sam_model_registry[model_type](checkpoint=sam_checkpoint)
     sam.to(device=device)

     # Existem diversas configurações para o uso da biblioteca!
     mask_generator_ = SamAutomaticMaskGenerator(
         model=sam,
         points_per_side=32,
         pred_iou_thresh=0.9,
         stability_score_thresh=0.96,
         crop_n_layers=1,
         crop_n_points_downscale_factor=2,
         min_mask_region_area=100,  # Requires open-cv to run
post-processing
     )

     masks = mask_generator_.generate(image)

     print(len(masks))
```

Código-fonte 6 - Usando a biblioteca SAM
Fonte: Elaborado pelo autor (2025)

E para exibir o resultado, precisamos do seguinte código:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Plotando tudo o que foi encontrado
    def show_anns(anns):
        if len(anns) == 0:
            return
        sorted_anns = sorted(anns, key=(lambda x: x['area']),
reverse=True)
        ax = plt.gca()
        ax.set_autoscale_on(False)
        polygons = []
        color = []
        for ann in sorted_anns:
            m = ann['segmentation']
            img = np.ones((m.shape[0], m.shape[1], 3))
            color_mask = np.random.random((1, 3)).tolist()[0]
            for i in range(3):
                img[:,:,i] = color_mask[i]
            ax.imshow(np.dstack((img, m*0.35)))

    plt.figure(figsize=(10,10))
    plt.imshow(image)
    show_anns(masks)
    plt.axis('off')
    plt.show()
```

Código-fonte 7 - Exibindo o resultado da segmentação pela biblioteca SAM
Fonte: Elaborado pelo autor (2025)

Esse código-fonte exige a imagem de entrada com uma camada sobreposta indicando as segmentações resultantes. Note pela figura a seguir que o método é tão poderoso que além de detectar um gato ou cachorro também pode identificar orelhas, olhos, patas, entre outros “objetos”:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 21 - Saída do código-fonte 7
Fonte: Elaborado pelo autor (2025)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A detecção e segmentação de objetos são pilares fundamentais da visão computacional, permitindo que máquinas identifiquem e diferenciem objetos em imagens e vídeos com alta precisão. Métodos tradicionais, como Haar Cascades e HOG + SVM, foram amplamente utilizados, mas suas limitações levaram ao desenvolvimento de redes neurais profundas especializadas, como Faster R-CNN, YOLO e Mask R-CNN. Essas abordagens modernas tornaram a detecção mais rápida, eficiente e precisa, sendo aplicadas em áreas como segurança, veículos autônomos, medicina e automação industrial.

As tendências futuras da segmentação de imagens incluem o desenvolvimento de modelos mais eficientes e acessíveis, que possam operar em dispositivos embarcados e smartphones sem perda de precisão. Além disso, espera-se um avanço na segmentação multimodal, combinando visão computacional com informações textuais e contextuais para melhorar a compreensão da cena. Modelos como o SAM indicam um futuro em que a segmentação será totalmente interativa e adaptável, possibilitando aplicações ainda mais avançadas em realidade aumentada, diagnóstico médico e inteligência artificial embarcada.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BOCHKOVSKIY, A.; WANG, C. Y.; LIAO, H. Y. M. **Yolov4**: Optimal speed and accuracy of object detection. 2020.

BOCHKOVSKIY, A.; LIAO, H. Y. M.; WANG, C. Y. YOLOv4: Optimal Speed and Accuracy of Object Detection. **arXiv**, 23 de abril de 2020. Disponível em: <https://arxiv.org/pdf/2004.10934>. Acesso em: 18 fev. 2025.

BONDARENKO, Kirill. Understanding UNET. **Medium**, 2 de julho de 2019. Disponível em: <https://bond-kirill-alexandrovich.medium.com/understanding-unet-27de538e08d8>. Acesso em: 18 fev. 2025.

BOURDAKOS, Nick. TensorFlow.js — Real-Time Object Detection in 10 Lines of Code. **Medium**, 10 de dezembro de 2018. Disponível em: <https://medium.com/hackernoon/tensorflow-js-real-time-object-detection-in-10-lines-of-code-baf15dfb95b2>. Acesso em: 18 fev. 2025.

HUGORUNE. HAAR Classifier Clarification. **Stack Overflow**, 30 de agosto de 2014. Disponível em: <https://stackoverflow.com/questions/25579225/haar-classifier-clarification>. Acesso em: 18 fev. 2025.

JIANG, Peiyuan, *et al*. A Review of Yolo algorithm developments. **Procedia computer science**, 2022. 199, p. 1066-1073.

KIRILLOV, Alexander, *et al*. **Segment anything**. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023. p. 4015-4026.

MEAN average precision (mAP) in object detection. **SuperAnnotate**, 12 de junho de 2023. Disponível em: <https://www.superannotate.com/blog/mean-average-precision-and-its-uses-in-object-detection>. Acesso em: 18 fev. 2025.

MINAEE, Shervin, *et al*. Image segmentation using deep learning: A survey. **IEEE transactions on pattern analysis and machine intelligence**, 2021, 44.7, p. 3523-3542.

MITTAL, Karthik. A Gentle Introduction Into The Histogram Of Oriented Gradients. **Medium**, 20 de dezembro de 2020. Disponível em: <https://medium.com/analytics-vidhya/a-gentle-introduction-into-the-histogram-of-oriented-gradients-fdee9ed8f2aa>. Acesso em: 18 fev. 2025.

NEVILLE. Dice Loss In Medical Image Segmentation. **Medium**, 21 de julho de 2023. Disponível em: <https://cvinvolution.medium.com/dice-loss-in-medical-image-segmentation-d0e476eb486>. Acesso em: 18 fev. 2025.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

RONNEBERGER, Olaf; FISCHER, Philipp; BROX, Thomas. U**-net: Convolutional networks for biomedical image segmentation**. In: Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18. Springer International Publishing, 2015. p. 234-241.

ROSEBROCK, Adrian. Intersection over Union (IoU) for object detection. **PyImageSearch**, 7 de novembro de 2016. Disponível em: <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>. Acesso em: 18 fev. 2025.

SHAKIBA, Amir. Let's look at Meta AI's SAM project and implement it with Python. **Medium**, 12 de abril de 2023. Disponível em: <https://medium.com/@amir_shakiba/lets-look-at-meta-ais-sam-project-and-implement-it-with-python-335926b21ed1>. Acesso em: 18 fev. 2025.

SIMIC, Milos. Intersection Over Union for Object Detection. **Baeldung**, 18 de março de 2024. Disponível em: <https://www.baeldung.com/cs/object-detection-intersection-vs-union>. Acesso em: 18 fev. 2025.

SKALSKI, Piotr. How to Use the Segment Anything Model (SAM). **Roboflow Blog**, 22 de janeiro de 2024. Disponível em: <https://blog.roboflow.com/how-to-use-segment-anything-model-sam/>. Acesso em: 18 fev. 2025.

THE evolution of YOLO: Object detection algorithms. **SuperAnnotate**, 25 de outubro de 2023. Disponível em: <https://www.superannotate.com/blog/yolo-object-detection>. Acesso em: 18 fev. 2025.

UMAM, Ardian. Introduction to How Faster R-CNN, Fast R-CNN and R-CNN Works. **YouTube**, 31 de março de 2017. Disponível em: <https://www.youtube.com/watch?v=v5bFVbQvFRk>. Acesso em: 18 fev. 2025.

WANG, Jo. R-CNN family explanation. **Medium**, 20 de outubro de 2024. Disponível em: <https://medium.com/@lokwa780/r-cnn-family-explanation-7a5070faa44c>. Acesso em: 18 fev. 2025.

YU, Ying, *et al.* **Techniques and challenges of image segmentation**: A review. Electronics, 2023. 12.5: p. 1199.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
CVAT	Computer Vision Annotation Tool
FCN	Fully Convolutional Networks
HOG	Histogram of Oriented Gradients
IoU	Intersection over Union
mAP	Mean Average Precision
NMS	Non-Maximum Suppression
R-CNN	Region-based Convolutional Neural Networks
RPN	Rede de Propostas de Região — Region Proposal Network
SAM	Segment Anything Model
SIFT	Scale-Invariant Feature Transform
SVM	Máquina de Vetores de Suporte
YOLO	You Only Look Once

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com